



Crime Count Forecasting per District

/ IT462

Student Name	IDs	Email
Raghad Almutairi	443200793	443200793@student.ksu.edu.sa
Sarah Alruwayte	444200758	444200758@student.ksu.edu.sa
Norah Alfaheed	444200779	444200779@student.ksu.edu.sa
Atheer Budie	444200894	444200894@student.ksu.edu.sa

Section: 2766
Supervised by: Dr. AFSHAN JAFRI

List of Contents

1	Introduction.....	2
2	Dataset Description.....	2
2.1	Main Attributes	3
2.2	Motivation.....	4
2.3	Reasons for choosing this dataset	4
3	Dataset Preprocessing	5
3.1	Data cleaning:	5
3.2	Data Integration:	9
3.3	Data Reduction:	10
3.4	Data Transformation:	12
4	Reference:	15

List of Tables

Table 1: Dataset attributes	4
-----------------------------------	---

1 Introduction

This project develops a regression-based crime count forecasting system that predicts the number of crime incidents in each Chicago district for future time periods using 971,865 historical crime records from 2021 to 2024. Instead of classifying locations as high risk or low risk, the system forecasts crime trends quantitatively at the district level using temporal and spatial patterns.

Accurate crime forecasting is important because it enables authorities to anticipate crime trends rather than react to past incidents. The system provides actionable insights for the Chicago Police Department, city officials, and urban planners to plan patrol deployments, allocate security resources, and evaluate policy effectiveness based on predicted crime volumes.

By predicting crime counts per district, the project supports proactive public safety planning, resource optimization, and evidence-based decision-making.

2 Dataset Description

Dataset Name: Crimes - 2001 to Present

Domain: Public Safety / Urban Analytics

Source: City of Chicago Data Portal

URL: <https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2>

Approximate Dataset Size:

- **Rows:** 971,865 records (2021-2024)
- **File Size:** 330 MB in CSV format
- **Number of Files/Tables:** 1 primary file

2.1 Main Attributes

The dataset contains **22 attributes** across the following types:

Attribute Type	Attribute Name	Description	Example/Range
Numeric	ID	Unique record identifier	12419690
Numeric	Beat	Police beat number	1-2535
Numeric	District	Police district number	1-31
Numeric	Ward	City ward number	1-50
Numeric	Community Area	Community area code	1-77
Numeric	X Coordinate	State plane X coordinate	1145000-1205000
Numeric	Y Coordinate	State plane Y coordinate	1850000-1950000
Numeric	Latitude	GPS latitude coordinate	41.6-42.1
Numeric	Longitude	GPS longitude coordinate	-87.9 to -87.5
Categorical	Primary Type	Main crime category	THEFT, BATTERY, ASSAULT, BURGLARY, ROBBERY (~30 categories)
Categorical	Description	Detailed crime subcategory	SIMPLE, AGGRAVATED, FROM BUILDING, etc.
Categorical	Location Description	Type of location where crime occurred	STREET, RESIDENCE, SIDEWALK, APARTMENT (~140 types)
Categorical	FBI Code	FBI crime classification code	01A, 04B, 06, etc.
Categorical	IUCR	Illinois Uniform Crime Reporting code	0460, 0820, 1310, etc.
Categorical	Arrest	Whether arrest was made	True/False
Categorical	Domestic	Whether incident was domestic-related	True/False
Temporal	Date	Incident occurrence timestamp	01/01/2021 12:00:00 AM
Temporal	Year	Year of incident	2021, 2022, 2023, 2024
Temporal	Updated On	Last update timestamp	02/08/2021 03:50:01 PM
Text	Case Number	Chicago Police Department case reference	JA123456
Text	Block	Partially anonymized address	001XX N STATE ST
Text	Description	Detailed crime description	SIMPLE, \$500 AND UNDER, etc.

Text	Location	Combined lat/long in point format	(41.88, -87.63)
------	----------	-----------------------------------	-----------------

Table 1: Dataset attributes

2.2 Motivation

Cities record millions of crime incidents each year, yet crime data is mostly used to describe past events rather than to anticipate future trends. Police departments typically respond after crimes occur because manually analyzing patterns across districts and time periods is impractical. This reactive approach limits efficient resource allocation and reduces the ability to prepare for fluctuations in crime levels.

The Chicago Crimes dataset contains large-scale historical records with spatial features (district, community area, coordinates), temporal features (hour, day, month, year), and detailed crime characteristics. These attributes enable the identification of temporal and geographic trends in crime activity over time.

Police departments require quantitative forecasts to support operational planning. Knowing approximately how many incidents to expect in each district during specific time periods allows for precise resource deployment, for example, anticipating 50 incidents versus 200 incidents in a district would significantly change staffing levels, patrol distribution, and response readiness. Numerical forecasting provides the granularity necessary for effective planning and evidence-based decision-making.

Therefore, this project aims to develop a district-level crime count forecasting system that transforms historical crime records into forward-looking insights, supporting proactive public safety planning rather than purely descriptive reporting.

2.3 Reasons for choosing this dataset

This dataset is ideal for Spark because its 971,865 records (330 MB) exceed the capacity of traditional single-machine processing tools and benefit from distributed computing. It includes rich spatial, temporal, and categorical features that support crime count forecasting per district over time. The dataset also introduces meaningful preprocessing challenges such as handling missing values, outliers, and encoding categorical variables. It enables efficient use of RDD operations and complex SQL queries for temporal pattern analysis and aggregation. Additionally, it supports regression modeling using Spark MLlib to predict future crime counts, demonstrating both Spark's technical capabilities and its practical application in addressing real-world public safety challenges. The dataset is publicly available, well-documented, and anonymized.

3 Dataset Preprocessing

3.1 Data cleaning:

- Dataset Size:**

To understand the scale of the dataset, we computed the total number of rows using Spark's `count()` function. The dataset contains 971,865 records.

```
scala> println("Rows: " + df.count())
Rows: 971865
```

- Missing Values Analysis:**

To assess data quality, we computed the number of missing values across all columns using a dynamic Spark aggregation approach. The results indicate that most columns contain no missing values, demonstrating overall data completeness. However, a few attributes contain null values, most notably latitude, longitude, x_coordinate, y_coordinate, and location (15,316 missing values each), location_description (4949 missing values), community_area (79 missing value), and ward (24 missing value). These missing values primarily relate to geographic or location-based attributes.

```
scala> val missLong=df.columns.map(c => (c, df.filter(col(c).isNull).count())) toSeq toDF("column","missing_nulls")
missLong: org.apache.spark.sql.DataFrame = [column: string, missing_nulls: bigint]

scala> missLong.orderBy(desc("missing_nulls")).show(50, false)
+-----+-----+
|column|missing_nulls|
+-----+-----+
|x_coordinate|15316|
|y_coordinate|15316|
|latitude|15316|
|longitude|15316|
|location|15316|
|location_description|4949|
|community_area|79|
|ward|24|
|id|0|
|case_number|0|
|date|0|
|block|0|
|lucr|0|
|primary_type|0|
|description|0|
|arrest|0|
|domestic|0|
|beat|0|
|district|0|
|fbi_code|0|
|year|0|
|updated_on|0|
+-----+-----+
```

Before deciding how to handle these missing records, we conducted an additional analysis to evaluate whether removing them would introduce bias into the dataset. Specifically, we examined the distribution of missing geographic values across key variables such as district, primary_type, year, domestic, and arrest status.

Missing Ratio by District:

The analysis of missing geographic records across districts shows that the missing ratio ranges between 1.3% and 3.57%. The highest proportion appears in District 11 (3.57%), followed by District 16 (3.52%), while several districts exhibit values close to 1%.

These results indicate that although slight variation exists across districts, the differences are relatively small and remain within a low percentage range (below 4%). Therefore, missing geographic information is not heavily concentrated in a specific district, and no strong geographic bias is observed.

```
scala> dfH.groupBy("district").agg(round(avg("isMissingAny")*100,3).alias("missing_ratio_pct")).orderBy(desc("missing_ratio_pct")).show(1000,false)
+-----+-----+
|district|missing_ratio_pct|
+-----+-----+
|11      |3.574            |
|16      |3.52             |
|31      |3.39             |
|19      |2.954            |
|18      |2.691            |
|1       |2.485            |
|20      |2.268            |
|14      |2.269            |
|12      |2.17             |
|24      |1.997            |
|17      |1.984            |
|22      |1.832            |
|2       |1.738            |
|10      |1.63             |
|25      |1.601            |
|15      |1.517            |
|8       |1.511            |
|5       |1.421            |
|4       |1.386            |
|9       |1.376            |
|6       |1.378            |
|7       |1.322            |
|3       |1.318            |
+-----+-----+
```

Missing Ratio by Primary Crime Type:

The distribution of missing records across crime categories shows variation, particularly among smaller crime classes.

For example:

- Human Trafficking: 18.868%
- Deceptive Practice: 12.573%
- Narcotics: 10.713%

However, these crime categories represent a very small proportion of the overall dataset (each contributing less than 3% individually in most cases).

In contrast, the dominant crime categories such as:

- Theft (21.994% of dataset) → 1.596% missing
- Battery (17.682%) → 0.460% missing
- Criminal Damage (11.421%) → 0.619% missing

show consistently low missing ratios (generally below 2%).

This indicates that missing values are primarily concentrated in relatively rare crime types, while the major crime categories that dominate the dataset remain largely unaffected. Therefore, the overall class distribution remains stable and the removal of missing records is unlikely to introduce significant bias into the modeling process.

```
scala> dfH.groupBy("primary_type").agg(count("*").alias("total_records"), round(count("isMissingAny")*100,3).alias("missing_ratio_pct"), round(count("*")/lit(totalRows)*100,3).alias("dataset_share_pct")).orderBy(desc("dataset_share_pct")).show(1000,false)
+-----+-----+-----+-----+
|primary_type|total_records|dataset_share_pct|missing_ratio_pct|
+-----+-----+-----+-----+
|THEFT       |213756      |21.994          |1.596            |
|BATTERY     |171844      |17.682          |0.46             |
|CRIMINAL DAMAGE|110994      |11.421          |0.619            |
|ASSAULT     |87269       |8.98            |0.433            |
|MOTOR VEHICLE THEFT|83045      |8.545          |0.448            |
|DECEPTIVE PRACTICE|68772      |7.076          |12.573           |
|OTHER OFFENSE|61434       |6.321          |1.467            |
|ROBBERY     |37049       |3.812          |0.232            |
|WEAPONS VIOLATION|34287      |3.52           |0.489            |
|BURGLARY    |30175       |3.185          |0.616            |
|NARCOTICS   |21619       |2.224          |10.713           |
|CRIMINAL TRESPASS|17382      |1.78           |0.613            |
|OFFENSE INVOLVING CHILDREN|7388      |0.752          |6.445            |
|CRIMINAL SEXUAL ASSAULT|6439      |0.663          |5.715            |
|SEX OFFENSE |4978        |0.512          |5.826            |
|PUBLIC PEACE VIOLATION|3156      |0.325          |0.665            |
|HOMICIDE    |2783        |0.286          |0.036            |
|INTERFERENCE WITH PUBLIC OFFICER|1983      |0.204          |0.303            |
|ARSON       |1946        |0.2            |0.36             |
|STALKING    |1872        |0.193          |2.831            |
|PROSTITUTION|894         |0.092          |0.783            |
|LIQUOR LAW VIOLATION|769       |0.079          |0.26             |
|CONCEALED CARRY LICENSE VIOLATION|751       |0.077          |0.666            |
|INTIMIDATION|709         |0.073          |2.257            |
|KIDNAPPING  |441         |0.045          |1.134            |
|OBSCENITY   |203         |0.021          |0.067            |
|GAMBLING    |59          |0.006          |0.0              |
|HUMAN TRAFFICKING|53         |0.005          |18.868           |
|PUBLIC INDECENCY|24         |0.002          |0.0              |
|NON-CRIMINAL|15          |0.002          |0.0              |
|OTHER NARCOTIC VIOLATION|16        |0.002          |0.0              |
+-----+-----+-----+-----+
```

Missing Ratio by Year:

Temporal analysis shows a clear downward trend: 2021 (3.651%), 2022 (2.41%), 2023 (1.288%), 2024 (0.955%). This demonstrates that missing geographic data significantly decreased over time. There is no abnormal spike in any specific year, suggesting that missing values are not tied to a particular temporal anomaly.

```
scala> dfM.withColumn("year", year(col("date"))).groupBy("year").agg(round(avg("isMissingAny")*100,3).alias("missing_ratio_pct")).orderBy("year").show(false)
+-----+-----+
|year|missing_ratio_pct|
+-----+-----+
|2021|3.651|
|2022|2.41|
|2023|1.288|
|2024|0.955|
+-----+-----+
```

Missing Ratio by Domestic Indicator:

Although slightly higher for non-domestic cases, the difference is small (less than 1%). This indicates no substantial domestic-related bias in missing geographic information.

```
scala> dfM.groupBy("domestic").agg(round(avg("isMissingAny")*100,3).alias("missing_ratio_pct")).show(false)
+-----+-----+
|domestic|missing_ratio_pct|
+-----+-----+
|true|0.96|
|false|2.232|
+-----+-----+
```

Missing Ratio by Arrest Status:

While arrest cases show a slightly higher missing percentage, the difference remains modest. The variation does not suggest systematic exclusion of a specific outcome group.

```
scala> dfM.groupBy("arrest").agg(round(avg("isMissingAny")*100,3).alias("missing_ratio_pct")).show(false)
+-----+-----+
|arrest|missing_ratio_pct|
+-----+-----+
|true|2.303|
|false|1.94|
+-----+-----+
```

Final Decision:

The results showed that missing values represent a small percentage of the overall dataset and are not disproportionately concentrated in major crime categories, dominant districts, or specific time periods. Although minor variation exists in some smaller categories, no substantial systematic bias was observed. Since the analysis did not reveal any significant bias associated with the missing geographic records, we proceeded with removing these incomplete observations. This decision ensures cleaner spatial-temporal aggregation while maintaining the integrity of the target variable (crime_count).

```
scala> val dfClean = df.na.drop(Seq(
  | "latitude",
  | "longitude",
  | "x_coordinate",
  | "y_coordinate",
  | "location",
  | "location_description",
  | "community_area",
  | "ward"
  | ))
dfClean: org.apache.spark.sql.DataFrame = [id: int, case_number: string ... 20 more fields]
```

- **Date Validation and Temporal Consistency:**

We did not perform a separate manual validation for invalid dates because the date column was successfully cast to Spark's timestamp type. According to Apache Spark's deterministic [1] timestamp parsing mechanism, any invalid or unparseable date values would automatically be converted to null rather than stored as incorrect dates. Since no null values were detected in the date column, this confirms that all stored dates are valid date values.

Additionally, to ensure temporal consistency with the defined study scope, we verified that all records fall within the expected analysis period (2021–2024). Using Spark's year() function, we filtered for any dates earlier than 2021 or later than 2024. The result showed 0 out-of-range records, confirming that all observations align with the specified time range of the study.

```
scala> val outOfRange = df.filter( year(col("date")) < 2021 || year(col("date")) > 2024).count()
outOfRange: Long = 0

scala> println("Out-of-range dates = " + outOfRange)
Out-of-range dates = 0
```

- **Duplicate Records:**

To detect duplicate records, we grouped the dataset by the unique identifier (id) and counted the occurrences of each ID. Any ID appearing more than once would indicate a duplicate record. The result showed 0 duplicated IDs, confirming that the dataset does not contain duplicate entries at the record level. Therefore, no duplicate removal step was required.

```
scala> val duplicateCount = df.groupBy("id").count().filter(col("count") > 1).count()
duplicateCount: Long = 0

scala> println("Number of duplicated IDs = " + duplicateCount)
Number of duplicated IDs = 0
```

- **District Value Validation:**

To ensure data consistency, we validated the district attribute against the officially defined police district boundaries published by the City of Chicago Open Data Portal (Boundaries – Police Districts) [2].

According to the official dataset, valid district identifiers range from 1 to 31.

We filtered the dataset to detect any values outside this range and found 0 records with invalid district values. Additionally, we extracted and reviewed all distinct district values in ascending order to verify that they fall within the expected domain.

This confirms that the district field contains only valid categorical values and does not require correction or cleaning.

```
scala> val invalidDistrict = df.filter(col("district") < 1 || col("district") > 31).count()
invalidDistrict: Long = 0

scala> println("District values outside range (1-31) = " + invalidDistrict)
District values outside range (1-31) = 0

scala> df.select("district").distinct().orderBy("district").show(50, false)
+-----+
|district|
+-----+
|1|
|2|
|3|
|4|
|5|
|6|
|7|
|8|
|9|
|10|
|11|
|12|
|14|
|15|
|16|
|17|
|18|
|19|
|20|
|22|
|24|
|25|
|31|
+-----+
```

Data Cleaning Summary:

Before the cleaning process, the dataset contained 971,865 records.

After removing rows that contained missing values the dataset size was reduced to 952,563 records.

A total of 19,302 records were removed during the cleaning process.

```
scala> val before = df.count()
before: Long = 971865

scala> val after = dfClean.count()
after: Long = 952563

scala> println("Removed rows: " + (before - after))
Removed rows: 19302
```

3.2 Data Integration:

The dataset used in this project consists of a single CSV file containing crime records in Chicago from 2021 to 2024.

Since the dataset is already consolidated into one source, no integration across multiple datasets or tables was required. Therefore, no join operations were performed.

Schema Documentation

Although no data integration was needed, the dataset schema was documented to ensure clarity and understanding of the data structure.

The schema was automatically inferred using Apache Spark. The attributes include different data types such as integers, strings, booleans, doubles, and timestamps.

Key attributes include:

- Temporal attributes: date, year, updated_on
- Spatial attributes: latitude, longitude, x_coordinate, y_coordinate
- Categorical attributes: primary_type, description, location_description
- Administrative attributes: district, ward, community_area
- Binary attributes: arrest, domestic

This schema supports temporal, spatial, and categorical analysis of crime data.

```
// Show schema
println("Schema of the Original Dataset:")
dfClean.printSchema()

Schema of the Original Dataset:
root
 |-- id: integer (nullable = true)
 |-- case_number: string (nullable = true)
 |-- date: timestamp (nullable = true)
 |-- block: string (nullable = true)
 |-- iucr: string (nullable = true)
 |-- primary_type: string (nullable = true)
 |-- description: string (nullable = true)
 |-- location_description: string (nullable = true)
 |-- arrest: boolean (nullable = true)
 |-- domestic: boolean (nullable = true)
 |-- beat: integer (nullable = true)
 |-- district: integer (nullable = true)
 |-- ward: integer (nullable = true)
 |-- community_area: integer (nullable = true)
 |-- fbi_code: string (nullable = true)
 |-- x_coordinate: integer (nullable = true)
 |-- y_coordinate: integer (nullable = true)
 |-- year: integer (nullable = true)
 |-- updated_on: timestamp (nullable = true)
 |-- latitude: double (nullable = true)
 |-- longitude: double (nullable = true)
 |-- location: string (nullable = true)

val cleanPath: String = /Users/raghadfares/Desktop/BigData_Phase2/clean_data.csv
val dfClean: org.apache.spark.sql.DataFrame = [id: int, case_number: string ... 20 more fields]
```

3.3 Data Reduction:

Data reduction was implemented to decrease both the volume and dimensionality of the dataset while preserving the critical spatial and temporal patterns required for accurate crime forecasting.

- **Sampling:** is typically used to reduce computational cost and speed up processing in large-scale datasets. However, it was not applied in this project because the forecasting task requires preserving complete temporal patterns across all districts and weeks. Removing a subset of records through random or stratified sampling could distort weekly crime trends and negatively affect model accuracy. Therefore, sampling was deemed unnecessary and no data was removed at this step, with the dataset remaining at 952,563 rows. This decision positively impacts the analysis by preserving full temporal information and improves model reliability by preventing information loss and biased trend representation.
- **Feature Selection:** To optimize computational efficiency and focus on predictive signals, we performed dimensionality reduction by retaining only the `date`, `district`, and `primary_type` attributes. While the forecasting target is the total crime count per district, `primary_type` was intentionally preserved to engineer high-value behavioral features. We removed 19 redundant or administrative attributes, such as `id`, `case_number`, and `block`, which do not contribute to long-term trend forecasting. Furthermore, detailed spatial features like `beat` and `ward` were excluded as the study is conducted specifically at the district level.

```
scala> // --- SECTION: DATA REDUCTION ---
// Step 1: Feature Selection
val dfSelected = dfClean.select("date", "district", "primary_type")
println("Step 1: Feature Selection Result")
dfSelected.show(10)
Step 1: Feature Selection Result
+-----+-----+-----+
|      date|district|primary_type|
+-----+-----+-----+
|2021-07-07 10:30:00|      5|SEX OFFENSE|
|2021-04-17 15:20:00|      6|ROBBERY|
|2021-09-08 16:45:00|     11|HOMICIDE|
|2021-02-22 13:36:00|     10|ROBBERY|
|2021-04-22 14:30:00|      7|DECEPTIVE PRACTICE|
|2021-09-01 09:00:00|     10|SEX OFFENSE|
|2021-05-05 15:31:00|     19|BATTERY|
|2021-05-30 13:45:00|     12|BATTERY|
|2021-06-11 10:44:00|     16|DECEPTIVE PRACTICE|
|2021-06-16 23:20:00|     24|BATTERY|
+-----+-----+-----+
only showing top 10 rows
val dfSelected: org.apache.spark.sql.DataFrame = [date: timestamp, district: int ... 1 more field]
```

- **Pivoted Aggregation:** The primary reduction technique used was aggregation, transforming the event-level dataset into a structured time-series format. We utilized Spark's `date_trunc` function to generate a `week_start` feature and performed a **pivoted aggregation** on the `primary_type` column. This allowed us to summarize individual incidents into weekly totals per district while maintaining the specific "crime mix" for each period.

```
scala> // Step 2: Pivoted Aggregation
val dfWeekly = dfSelected
  .withColumn("week_start", date_trunc("week", col("date")))
  .groupBy("district", "week_start")
  .pivot("primary_type")
  .count()
  .na.fill(0)

println("Step 2: Pivoted Aggregation Result")
dfWeekly.select("district", "week_start", "THEFT", "BATTERY", "ASSAULT").show(10, truncate = false)
Step 2: Pivoted Aggregation Result
26/03/05 13:24:11 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
+-----+-----+-----+-----+
|district|week_start|THEFT|BATTERY|ASSAULT|
+-----+-----+-----+-----+
|4|2021-10-18 00:00:00|29|54|25|
|25|2021-10-18 00:00:00|43|42|20|
|1|2022-07-11 00:00:00|106|28|28|
|4|2022-08-08 00:00:00|52|48|33|
|5|2022-06-13 00:00:00|26|38|15|
|1|2022-10-10 00:00:00|90|26|9|
|14|2022-10-10 00:00:00|64|20|11|
|20|2023-12-18 00:00:00|24|16|11|
|24|2023-10-09 00:00:00|41|39|11|
|7|2023-08-21 00:00:00|21|55|16|
+-----+-----+-----+-----+
only showing top 10 rows
val dfWeekly: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 31 more fields]
```

- **Quantitative Impact:** This step resulted in a significant reduction in data volume, moving from **952,563 incident records** to **4,667 aggregated weekly rows**, a reduction of approximately **99.5%**. This transition from granular noise to structural trends improves model stability and significantly enhances scalability within the Spark environment.

```
scala> // Step 3: Target Variable Definition
val crimeTypeCols = dfWeekly.columns.diff(Seq("district", "week_start"))
val dfWithTarget = dfWeekly.withColumn("total_crime_count",
  crimeTypeCols.map(col).reduce(_ + _))

println("Step 3: Target Variable Calculation")
dfWithTarget.select("district", "week_start", "total_crime_count").show(10)
Step 3: Target Variable Calculation
+-----+-----+-----+
|district|week_start|total_crime_count|
+-----+-----+-----+
|4|2021-10-18 00:00:00|229|
|25|2021-10-18 00:00:00|206|
|1|2022-07-11 00:00:00|285|
|4|2022-08-08 00:00:00|201|
|5|2022-06-13 00:00:00|186|
|1|2022-10-10 00:00:00|244|
|14|2022-10-10 00:00:00|187|
|20|2023-12-18 00:00:00|186|
|24|2023-10-09 00:00:00|165|
|7|2023-08-21 00:00:00|201|
+-----+-----+-----+
only showing top 10 rows
val crimeTypeCols: Array[String] = Array(ARSON, ASSAULT, BATTERY, BURGLARY, CONCEALED CARRY LICENSE VIOLATION, CRIMINAL DAMAGE, CRIMINAL SEXUAL ASSAULT, CRIMINAL TRESPASS, DECEPTIVE PRACTICE, GAMBLING, HOMICIDE, HUMAN TRAFFICKING, INTERFERENCE WITH PUBLIC OFFICER, INTIMIDATION, KIDNAPPING, LIQUOR LAW VIOLATION, MOTOR VEHICLE THEFT, NARCOTICS, NON-CRIMINAL, OBSCENITY, OFFENSE INVOLVING CHILDREN, OTHER NARCOTIC VIOLATION, OTHER OFFENSE, PROSTITUTION, PUBLIC INDECENCY, PUBLIC PEACE VIOLATION, ROBBERY, SEX OFFENSE, STALKING, THEFT, WEAPONS VIOLATION)
val dfWithTarget: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 32 more fields]
scala>
```

Note on Feature Preservation: Although the final objective is to forecast the `total_crime_count`, we intentionally preserved individual crime categories through a pivoted aggregation. This decision was made to provide the regression model with "behavioral features," allowing the algorithm to identify if specific distributions of crime types serve as leading indicators for overall crime volume fluctuations in subsequent periods.

Comparison of Dataset Size Before and After Data Reduction

The original dataset contained **971,865 records and 22 columns**. After applying the data reduction process, which involved aggregating crimes by **district and week** and using a pivot operation on the **primary crime type**, the dataset size changed significantly.

The number of **rows decreased to 4,667**, because multiple individual crime records were summarized into weekly totals for each district. This aggregation reduced the dataset volume while preserving the essential spatial and temporal crime patterns needed for analysis.

At the same time, the number of **columns increased to 33**, since the pivot operation transformed each crime type into a separate numerical feature representing the weekly count of that crime type.

Overall, the reduction process **greatly decreased the number of rows while slightly increasing the number of columns**, producing a more compact and analysis-ready dataset suitable for machine learning.

```
scala> println("Dataset size after reduction:")
| println("Number of rows after reduction: " + dfWeekly.count())
| println("Number of columns after reduction: " + dfWeekly.columns.length)
Dataset size after reduction:
Number of rows after reduction: 4667
Number of columns after reduction: 33
```

3.4 Data Transformation:

Data transformation was performed to prepare the reduced dataset for analysis and ensure compatibility with Spark MLlib algorithms.

- **Temporal Feature Engineering:** Because crime in Chicago is highly seasonal, we extracted `month` (1–12) and `week_no` (1–52) from the `week_start` attribute. These features allow the model to capture annual fluctuations, such as the typical increase in certain crime types during winter holidays or summer months.


```
scala> // --- SECTION: DATA TRANSFORMATION ---
// Step 4: Temporal Feature Engineering
val dfTransformed = dfWithTarget
  .withColumn("month", month(col("week_start")))
  .withColumn("week_no", weekofyear(col("week_start")))

println("Step 4: Temporal Features (Month/Week)")
dfTransformed.select("week_start", "month", "week_no").show(10)
Step 4: Temporal Features (Month/Week)
+-----+-----+-----+
| week_start | month | week_no |
+-----+-----+-----+
| 2021-10-18 00:00:00 | 10 | 42 |
| 2021-04-05 00:00:00 | 4 | 14 |
| 2021-06-14 00:00:00 | 6 | 24 |
| 2021-01-11 00:00:00 | 1 | 2 |
| 2021-04-19 00:00:00 | 4 | 16 |
| 2021-04-12 00:00:00 | 4 | 15 |
| 2021-05-31 00:00:00 | 5 | 22 |
| 2021-07-12 00:00:00 | 7 | 28 |
| 2021-04-19 00:00:00 | 4 | 16 |
| 2021-11-01 00:00:00 | 11 | 44 |
+-----+-----+-----+
only showing top 10 rows
val dfTransformed: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 34 more fields]
```

- Categorical Encoding:** We applied the `StringIndexer` estimator to the `district` attribute. Although districts are represented by numeric identifiers (1–31), they are fundamentally categorical. Encoding these as indices ensures that the machine learning models in Phase 5 treat each district as a unique category rather than a continuous numeric variable.

```
scala> // Step 5: Categorical Encoding
val indexer = new StringIndexer()
  .setInputCol("district")
  .setOutputCol("district_indexed")
val dfIndexed = indexer.fit(dfTransformed).transform(dfTransformed)

println("Step 5: StringIndexer Result")
dfIndexed.select("district", "district_indexed").distinct().orderBy("district_indexed").show(10)
Step 5: StringIndexer Result
+-----+-----+
| district | district_indexed |
+-----+-----+
| 1 | 0.0 |
| 10 | 1.0 |
| 11 | 2.0 |
| 12 | 3.0 |
| 14 | 4.0 |
| 15 | 5.0 |
| 16 | 6.0 |
| 17 | 7.0 |
| 18 | 8.0 |
| 19 | 9.0 |
+-----+-----+
only showing top 10 rows
val indexer: org.apache.spark.ml.feature.StringIndexer = strIdx_7f322906d0e3
val dfIndexed: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 35 more fields]
```

- Domain-Specific Lag Engineering:** A critical transformation for time-series forecasting was the creation of a **Lag Feature** (`prev_week_total`). Using Spark Window functions partitioned by district, we captured the total crime count from the immediately preceding week. **Rationale:** In urban analytics, historical momentum is the strongest predictor of future activity. By providing the model with the previous week's performance, we enable the algorithm to understand current trends rather than relying solely on static averages.

```
scala> // Step 6: Domain-Specific Lag Engineering
      | val windowSpec = Window.partitionBy("district").orderBy("week_start")
      | val dfFinal = dfIndexed.withColumn("prev_week_total", lag("total_crime_count", 1).over(windowSpec))
      | .na.drop()
      |
      | println("Step 6: Lag Feature Result")
      | dfFinal.select("district", "week_start", "total_crime_count", "prev_week_total").show(10)
Step 6: Lag Feature Result
+-----+-----+-----+-----+
|district|week_start|total_crime_count|prev_week_total|
+-----+-----+-----+-----+
|1|2021-01-04 00:00:00|123|47|
|1|2021-01-11 00:00:00|124|123|
|1|2021-01-18 00:00:00|118|124|
|1|2021-01-25 00:00:00|96|118|
|1|2021-02-01 00:00:00|104|96|
|1|2021-02-08 00:00:00|100|104|
|1|2021-02-15 00:00:00|104|100|
|1|2021-02-22 00:00:00|133|104|
|1|2021-03-01 00:00:00|127|133|
|1|2021-03-08 00:00:00|107|127|
+-----+-----+-----+-----+
only showing top 10 rows
val windowSpec: org.apache.spark.sql.expressions.WindowSpec = org.apache.spark.sql.expressions.WindowSpec@ee2702a
val dfFinal: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 36 more fields]
```

Planned Numerical Scaling and Strategic Deferral:

To prepare the dataset for predictive modeling, we identified numerical features that require scaling and defined a strategy to apply this step while maintaining data integrity.

- **Features to be Scaled:** The numerical features selected for scaling are **prev_week_total**, **month**, and **week_no**.
- **Rationale for Scaling:** These features have different numerical ranges. Since we plan to use gradient-based models in **Spark MLlib** (e.g., Linear Regression), scaling is necessary to ensure that features with larger values do not dominate the model and bias the learned weights.
- **Implementation Strategy:** Although scaling is an important preprocessing step, its implementation has been intentionally postponed to **Phase 5: Machine Learning**.
- **Avoiding Data Leakage:** Applying scaling to the entire dataset at this stage could lead to **data leakage**, where information from the test data influences the training process through global statistics such as the mean or standard deviation.
- **Methodological Integrity:** To prevent this issue, **StandardScaler** will be included within a **Spark ML Pipeline** during the modeling phase. This ensures that the scaler is fitted only on the training data and then applied to the test data, preserving the reliability of the model evaluation.

Final preprocessed dataset:

```
scala> // Selecting a balanced mix of spatial, temporal, behavioral, and engineered features
val finalSnapshot = dfFinal.select(
  "district",
  "week_start",
  "month",
  "week_no",
  "THEFT",
  "BATTERY",
  "NARCOTICS",
  "total_crime_count",
  "prev_week_total"
)

println("Final Preprocessed Dataset Snapshot (15 rows):")
finalSnapshot.show(15, truncate = false)
Final Preprocessed Dataset Snapshot (15 rows):
```

	district	week_start	month	week_no	THEFT	BATTERY	NARCOTICS	total_crime_count	prev_week_total
1	1	2021-01-04 00:00:00	1	1	39	17	3	123	47
1	1	2021-01-11 00:00:00	1	2	39	18	0	124	123
1	1	2021-01-18 00:00:00	1	3	39	13	2	118	124
1	1	2021-01-25 00:00:00	1	4	21	18	0	96	118
1	1	2021-02-01 00:00:00	2	5	27	13	1	104	96
1	1	2021-02-08 00:00:00	2	6	28	25	1	100	104
1	1	2021-02-15 00:00:00	2	7	29	17	0	104	100
1	1	2021-02-22 00:00:00	2	8	40	16	0	133	104
1	1	2021-03-01 00:00:00	3	9	26	30	1	127	133
1	1	2021-03-08 00:00:00	3	10	29	17	1	107	127
1	1	2021-03-15 00:00:00	3	11	35	23	1	115	107
1	1	2021-03-22 00:00:00	3	12	30	20	1	114	115
1	1	2021-03-29 00:00:00	3	13	20	31	2	111	114
1	1	2021-04-05 00:00:00	4	14	40	25	1	136	111
1	1	2021-04-12 00:00:00	4	15	33	22	2	129	136

```
only showing top 15 rows
val finalSnapshot: org.apache.spark.sql.DataFrame = [district: int, week_start: timestamp ... 7 more fields]
```

4 Reference:

- [1] B. Chambers and M. Zaharia, 'Spark: The definitive guide', *Wordpress.com*. [Online]. Available: https://analyticsdata24.wordpress.com/wp-content/uploads/2020/02/spark-the-definitive-guide40www.bigdatabugs.com_.pdf. [Accessed: 26-Feb-2026].
- [2] 'Boundaries - police districts (current)', *Cityofchicago.org*. [Online]. Available: <https://data.cityofchicago.org/Public-Safety/Boundaries-Police-Districts-current-/fthy-xz3r>. [Accessed: 27-Feb-2026].